# Simulator 4: Developer Productivity with Claude

Alignment: Domains 2, 3 and 1 · Runs on the Claude Agent SDK through your Claude login, against mock backends

## Objective

Drive a real Agent SDK session through a legacy billing repo the way the exam expects: consult an MCP catalogue before touching files, Grep before Read, push the noisy caller search into an Explore-style subagent, keep a scratchpad, resume the same session after a teammate changed a file, fix a rounding defect with Edit (falling back to Read plus Write when the anchor is not unique), prove it with pytest, fork the session to compare two designs, and recover a crashed session from a manifest without re-exploring.

## Scenario (official exam scenario 4)

"You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers."

Primary domains: Tool Design & MCP Integration, Claude Code Configuration & Workflows, Agentic Architecture & Orchestration.

## Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

## What is in the folder

| File | Role |
|---|---|
| `explorer.py` | Four phases: explore, edit (resume), fork (two branches), recover (fresh session from the manifest); hooks; MCP catalogue server |
| `mcp_resources.json` | The `orderly-docs` catalogue the agent must read before exploring |
| `sample_repo/` | `src/billing/invoice.py` (the unrounded total), a validation module, a legacy wrapper, two pytest files, two docs |
| `cases/UC-*.json` | Seven use cases (Q37 plus six task-statement drills): situation, correct answer, prompt template, rubric |
| `verify.py` | Twelve acceptance checks over `evidence/`, plus a summary of any use-case results |
| `evidence/` | `tool_log.jsonl`, `sessions.json`, one JSON per phase, `scratchpad.md`, `manifest.json`, `patched-invoice.py.txt` |

© 2026 Tertiary Infotech Academy · UEN 201200696W

## Guided tour

**1. Read the defect and the two hooks. `calculate_total` returns an unrounded Decimal; `return total` appears twice in the file. The PostToolUse hook logs every call; the PreToolUse hook denies any Bash command containing `rm -rf`.**

```
cat sample_repo/src/billing/invoice.py
sed -n '/^async def log_tool/,/^HOOKS/p' explorer.py
```

**2. Run all four phases. Each prints its result line with turns, cost and session id.**

```
uv run explorer.py
```

**3. Read the explore tool log in order. The catalogue call comes first, a Grep precedes the first Read, and the caller search ran inside the `explore` subagent (its rows carry `"agent": "explore"`).**

```
grep '"phase": "explore"' evidence/tool_log.jsonl | python3 -c "import sys,json;
    [print(json.loads(l)['agent'], json.loads(l)['tool'], json.loads(l)['input']) for l in
    sys.stdin]"
```

**4. Open the scratchpad the agent kept in `NOTES.md`. It names the definition site, the callers and the proposed fix, so a later session can start from facts instead of re-reading.**

```
cat evidence/scratchpad.md
```

**5. Confirm the edit phase resumed the same session (`sessions.json` shows one id for explore and edit) and was told exactly which file changed. Then look for the Edit call and any non-unique-anchor failure or Write fallback.**

```
cat evidence/sessions.json
grep '"phase": "edit"' evidence/tool_log.jsonl | grep -E '"Edit"|"Write"|"Bash"'
```

**6. Check the patched file and run the tests yourself on the work copy.**

```
grep -n quantize evidence/patched-invoice.py.txt
cd work && python3 -m pytest -q && cd ..
```

**7. Compare the two forks. Both start from the explore history, get new session ids, and never touch the original; read their eight-line assessments.**

```
python3 -c "import json; [print(p, json.load(open(f'evidence/{p}.json'))['result'][:600],
    sep='\n') for p in ('fork-a','fork-b')]"
```

**8. Read the manifest and the recovery run. The fresh session got the manifest in its prompt and finished without a single Grep or subagent.**

```
cat evidence/manifest.json
grep '"phase": "recover"' evidence/tool_log.jsonl
```

**9. Run the verifier.**

```
uv run verify.py
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

**10. Stretch: ask the recover phase to `rm -rf work/tests` and watch the PreToolUse hook deny it in `tool_log.jsonl` (`"blocked"` row).**

## Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 4. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

| Lab | Task | Source | What it proves | Command |
|---|---|---|---|---|
| 14 | 1.5 | Task statement | rm -rf meets a hook, not a polite request | `uv run explorer.py --case UC-D6` |
| 17 | 1.7 | Task statement | Resume, and say exactly which file changed | `uv run explorer.py --case UC-D4` |
| 24 | 2.4 | Task statement | Read the catalogue before opening a single file | `uv run explorer.py --case UC-D2` |
| 25* | 2.5 | Practice Q37 | /find-callers needs Grep with a glob, not Bash | `uv run explorer.py --case UC-37` |
| 26 | 2.5 | Task statement | Edit says the anchor isn't unique. Now what? | `uv run explorer.py --case UC-D1` |
| 56* | 5.4 | Task statement | Delegate the noisy search, keep a scratchpad | `uv run explorer.py --case UC-D3` |
| 58 | 5.4 | Task statement | Crash recovery from a manifest | `uv run explorer.py --case UC-D5` |

## Expected evidence

- `evidence/tool_log.jsonl`: every tool call across the four phases with agent and phase labels.
- `evidence/cases/UC-<id>.json`: one record per use case with the filled prompt, session id, cost, final text and rubric results.
- `evidence/sessions.json`: explore and edit share an id; fork-a, fork-b and recover have their own.
- `evidence/scratchpad.md`, `evidence/manifest.json`, `evidence/patched-invoice.py.txt`.
- `evidence/verification.txt`: PASS lines from step 9.

## Acceptance checks

- Catalogue before Read; Grep before the main agent's first Read; the caller search delegated.
- Edit session id equals explore session id; Edit was attempted and the fix (`quantize`, `0.01`) is in place; pytest passes.
- Two forks with new ids; recovery session has no Grep and spawns no subagent.
- The pristine `sample_repo/` is unchanged.

## Exam link

Task statements 2.5 (Grep then Read, Edit fallback to Read plus Write), 2.4 (MCP catalogue so the agent sees what exists without exploring), 1.5 (PreToolUse interception, PostToolUse logging), 1.7 (`resume`, `fork_session`, telling a resumed session what changed), 3.4 (Explore-style delegation), 5.4 (scratchpad, manifest-based crash recovery).

© 2026 Tertiary Infotech Academy · UEN 201200696W